ROS2

Tanmayee

Driverless Taskphase – Task 3

Code: `https://github.com/tanmayee000/Tanmayee_Driverless_Taskphase` (folder `Task3_ROS2`)

***Abstract*—This report covers the installation of ROS 2 Humble on Ubuntu 22.04 and its basic ideas: nodes, topics, services, actions, parameters, logs, launch files, custom messages, colcon, workspaces and QoS. Two problems are then solved. In the first, one node takes a wheel's angular velocity and radius from the user and sends them in a custom `TaskOne` message, and another node calculates the wheel's speed using $v = \omega r$. In the second, three nodes work together: one sends strings, one checks if each string is a palindrome, and one replies "Yes" or "No". The report shows the output and ROS graphs for both.**

***Index Terms*—ROS 2, topics, services, custom messages, colcon, QoS.**

I. Introduction

ROS 2 (Robot Operating System 2) is software that helps the different parts of a robot talk to each other. Instead of writing one huge program, the robot's work is split into many small programs, and ROS 2 lets them send data to each other. A driverless car works this way: separate programs handle the cameras, finding the car's position, planning the path and steering, and they all need to share data quickly and reliably.

The task was done in three parts: (i) installing Ubuntu 22.04 and ROS 2 Humble, (ii) following the official beginner tutorials, and (iii) writing two solutions in Python. The next sections explain the main ROS 2 ideas with screenshots from the tutorials, and the last sections explain the two solutions.

II. Setup

Ubuntu 22.04 was run inside a virtual machine using Parallels Desktop on a Mac. ROS 2 Humble was then installed by following the official installation guide. To check that it worked, the `talker` and `listener` demo programs were run: the talker sends "Hello World" messages and the listener receives them (Fig. 1).

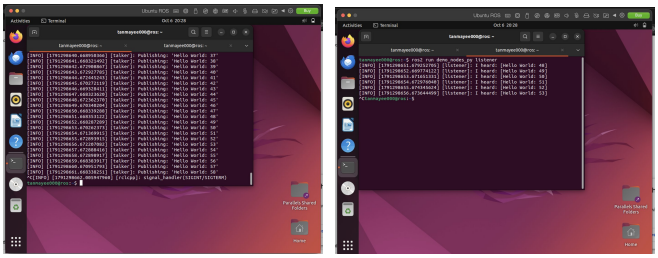


Fig. 1. Testing the installation: the `talker` (left) sends messages and the `listener` (right) receives them.

III. The ROS 2 Computation Graph

The *computation graph* is the full picture of all the programs (nodes) that are running and how they are connected. It can be drawn with the tool `rqt_graph`. In this drawing, ovals are nodes, rectangles are topics, and arrows show which way the data goes.

Fig. 2 shows the graph from the turtlesim tutorial. The `/teleop_turtle` node reads the keyboard and sends movement commands on `/turtle1/cmd_vel`, and the `/turtlesim` node receives them and moves the turtle. The other two arrows belong to the turtle's rotate action (Section VI).

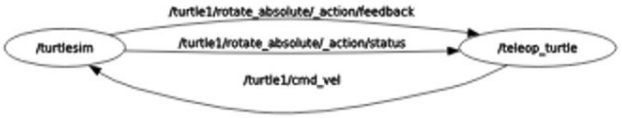


Fig. 2. `rqt_graph` of turtlesim being controlled from the keyboard.

IV. Nodes

A *node* is one small program that does one job, for example reading a sensor or controlling a wheel. A robot is made of many nodes working together. Useful commands:

- `ros2 run <package> <program>` starts a node.
- `ros2 node list` shows all running nodes.
- `ros2 node info <node>` shows what a node sends, receives and offers (Fig. 3).

A node can also be given a new name when it starts, without changing its code. This is called *remapping*. It was used to run a second turtlesim called `my_turtle`.

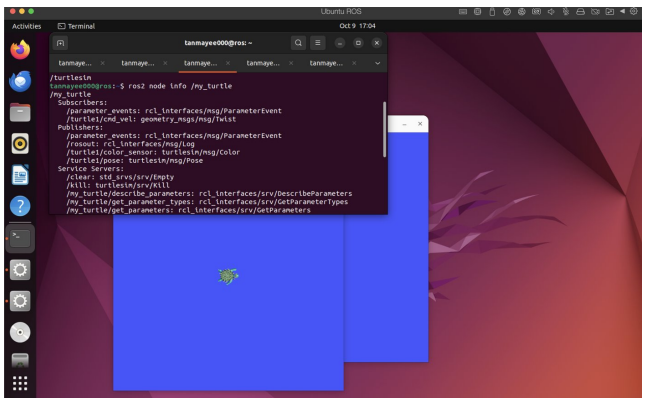


Fig. 3. `ros2 node info /my_turtle` lists what the node sends and receives.

V. Topics

A *topic* is like a radio channel. One node *publishes* (sends) messages on it, and any node that *subscribes* (listens) receives them. The sender does not know who is listening and does not get a reply. Each topic carries one type of message. For example, `/turtle1/cmd_vel` carries a `Twist` message, which holds a forward speed and a turning speed.

The commands `ros2 topic list`, `echo` (watch mes-

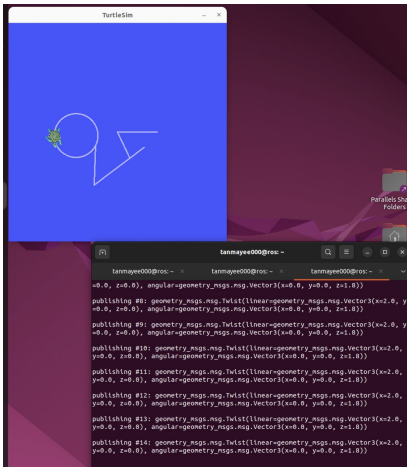


Fig. 4. Publishing speed commands with `ros2 topic pub`; the turtle moves in a circle.

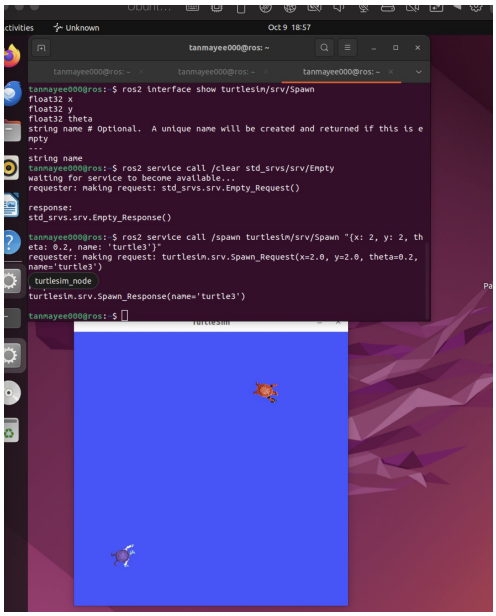


Fig. 5. Calling the `/clear` and `/spawn` services from the terminal.

sages), `info`, `hz` (how often messages arrive) and `pub` (send a message by hand) were tried. Sending a forward speed and a turning speed once every second made the turtle drive in a circle (Fig. 4). Topics are used for data that keeps coming, like sensor readings or speed commands.

VI. Services, Actions and Parameters

A. Services

A *service* is like a phone call. One node (the *client*) asks a question, and another node (the *server*) sends back one answer. It only happens when the client asks. A service file has two parts separated by `---`: the question (request) on top and the answer (response) below. The `/clear` service was used to erase the turtle's lines and `/spawn` was used to add a new turtle (Fig. 5). rqt's Service Caller was also used to make the turtle draw a thick red line with `/turtle1/set_pen` (Fig. 6).

B. Actions

An *action* is for a task that takes some time. It is like ordering food: you place the order (the *goal*), get updates while you

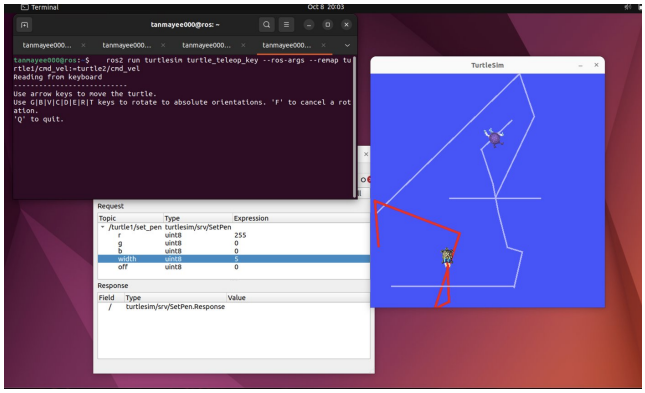


Fig. 6. Using rqt's Service Caller to change the pen to red with width 5.

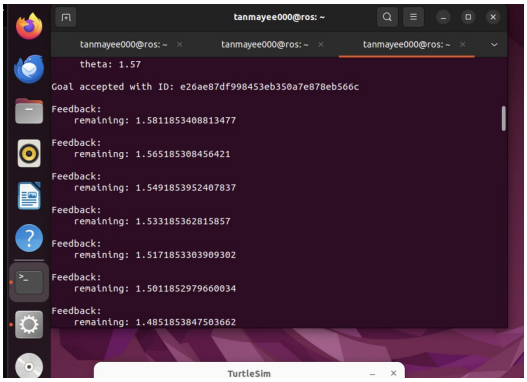


Fig. 7. Sending a rotate goal; the feedback shows how much turning is left.

wait (the *feedback*), and finally get the food (the *result*). An action can also be cancelled halfway. The turtle was asked to rotate to 1.57 radians (90°), and the feedback showed the remaining angle getting smaller until it finished (Fig. 7).

C. Parameters

A *parameter* is a setting inside a node, like a volume knob, that can be read or changed while the node is running. The commands `ros2 param list`, `get` and `set` were used. Changing `background_r` to 150 turned the turtlesim background purple straight away (Fig. 8). Settings can also be saved to a file and loaded again later.

D. Logs and Launch Files

Nodes print messages called *logs*, which have five levels: Fatal, Error, Warn, Info and Debug. `rqt_console` shows the logs from all nodes in one window. When the turtle hit the wall, it showed a Warn message, "Oh no! I hit the wall!" (Fig. 9). A *launch file* starts many nodes with one command. The `multisim` launch file opened two turtlesims at once, and each turtle was made to move in its own circle (Fig. 10).

VII. Topic vs. Service

A **topic** sends data in one direction and does not wait for a reply. It is good for data that keeps coming, can go to many listeners, and needs no answer, like camera images, sensor readings and speed commands.

A **service** sends a question and waits for one answer. It is good for things that happen once in a while where the sender needs a result, like resetting the simulator, adding a turtle, or (in Q2) asking another node for a "Yes" or "No".

For long tasks that need progress updates or might need

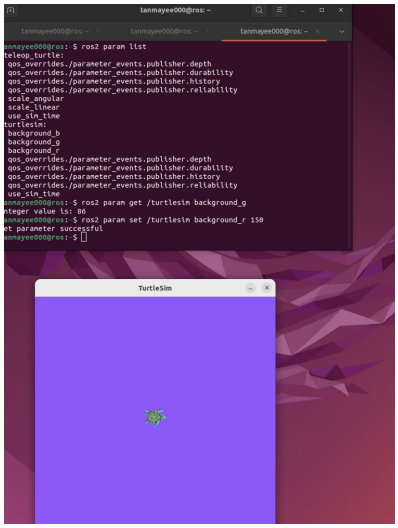


Fig. 8. Changing the background colour with `ros2 param set`.

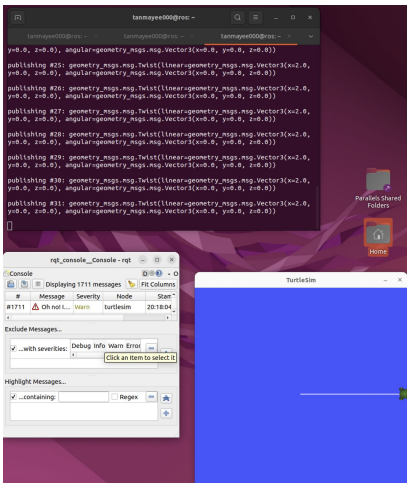


Fig. 9. `rqt_console` showing a warning when the turtle hits the wall.

cancelling, an **action** is better than either. Table I sums up the differences.

VIII. Custom Interfaces

Sometimes the built-in message types don't fit, so you can make your own: messages (`.msg`), services (`.srv`) and actions (`.action`). These must go in a CMake-type package. The file `CMakeLists.txt` lists them so ROS 2 can turn them into code that any node can import. One package, `task_interfaces`, holds all three files:

```
# msg/TaskOne.msg
float64 ang_vel
float64 radius
# msg/RT2String.msg
string data
# srv/PalindromeResult.srv
int32 is_palindrome
---
string answer
```

ROS 2 does not allow capital letters in field names, so the field `angVel` was written as `ang_vel`. Fig. 11 shows the message after building.

IX. colcon and Workspace Structure

A *workspace* is the folder where ROS 2 code lives. Packages are kept in the `src/` folder. Running `colcon build`

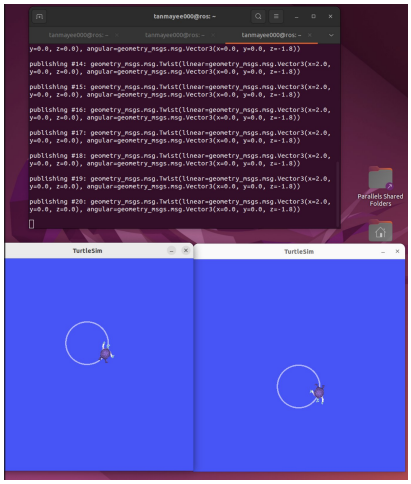


Fig. 10. One launch file starting two turtlesims.

TABLE I
TOPIC VS. SERVICE

	Topic	**Service**
Works like	Radio broadcast	Phone call
Direction	One way	Question and answer
Reply	None	One per question
When	All the time	Only when asked
File type	`.msg`	`.srv`
Example	Sensor data, speed	Reset, spawn, queries

builds every package and creates three more folders (Fig. 12):

- `build/`: temporary files used while building.
- `install/`: the finished programs, ready to run.
- `log/`: a record of each build.

After building, `source install/setup.bash` tells the terminal where the built programs are. colcon also builds packages in the right order, so the message package is built before the packages that use it.

The main ROS 2 installation is called the *underlay*, and a workspace on top of it is the *overlay*. If the same package is in both, the overlay's version is used. To test this, turtlesim's window title was changed in the workspace. With the workspace loaded it opened as "MyTurtleSim", and without it the normal "TurtleSim" opened (Fig. 13).

X. Quality of Service (QoS)

QoS settings decide *how* messages are delivered between a publisher and a subscriber [1]. The main ones are:

- **History and depth:** how many messages to keep in the queue (for example, the last 10).
- **Reliability:** *reliable* keeps resending until the message arrives; *best effort* may drop messages but is faster.
- **Durability:** whether a node that joins late gets the last message (*transient local*) or not (*volatile*).
- **Deadline, lifespan and liveliness:** how often messages should arrive, how long they stay valid, and how to tell if a publisher is still alive.

Fast sensor data such as camera images usually uses best effort, because a new image is coming anyway. The publisher's and subscriber's settings must match. For example, a reliable subscriber cannot connect to a best-effort publisher, and when this happens nothing is received and no error is shown. Both solutions use the default settings (reliable, keep last

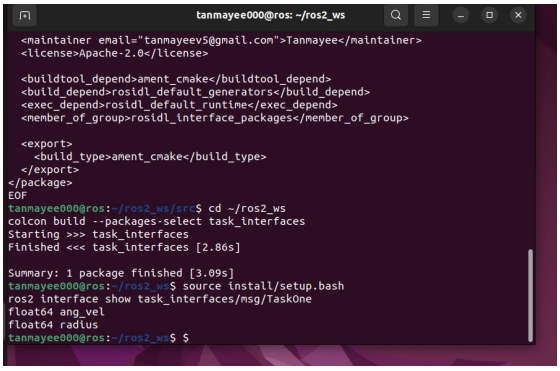


Fig. 11. Building `task_interfaces` and checking the `TaskOne` message.

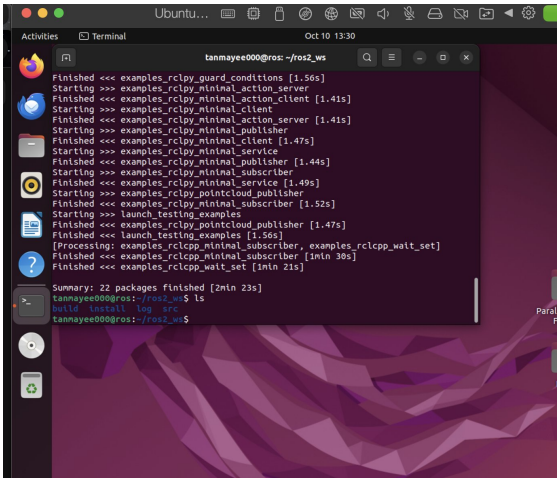


Fig. 12. `colcon build` and the workspace folders `build`, `install`, `log` and `src`.

10). That is the `10` at the end of `create_publisher` and `create_subscription`.

XI. Solution 1: Wheel Speed

A. Problem and Design

One node asks the user for a wheel's angular velocity ω and radius r and sends them on the topic `/taskone` in a `TaskOne` message. A second node receives them and prints the wheel's forward speed:

$$v = \omega \times r \qquad (\text{rad/s} \times \text{m} = \text{m/s}). \tag{1}$$

The data only goes one way and no reply is needed, so a topic is used. Two packages are used: `task_interfaces` for the message and `task_one` for the two nodes, `wheel_publisher` and `speed_subscriber`.

B. Code

The publisher keeps asking for input, checks that the user typed numbers and that the radius isn't negative, and then sends the message. The subscriber runs a function every time a message arrives:

```
self.create_subscription(TaskOne, '/taskone',
                         self.listener_callback, 10)
def listener_callback(self, msg):
    speed = msg.ang_vel * msg.radius
    self.get_logger().info(f'... v = w x r = {speed:.3f}
     m/s')
```

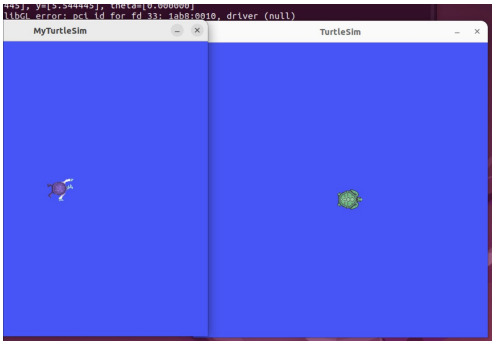


Fig. 13. Edited turtlesim (left, overlay) next to the original (right, underlay).

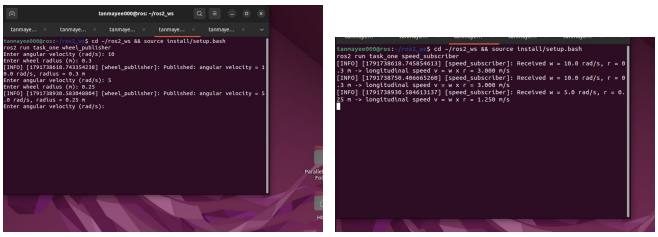


Fig. 14. Solution 1: the publisher takes input (left) and the subscriber prints the speed (right).

C. Results

Entering $\omega = 10$, $r = 0.3$ gave 3.000 m/s, and $\omega = 5$, $r = 0.25$ gave 1.250 m/s, which are correct (Fig. 14). The ROS graph shows the publisher, the topic and the subscriber connected (Fig. 15).

XII. Solution 2: Palindrome Checker

A. Problem and Design

Node 1 sends strings typed by the user in an `RT2String` message. Node 2 checks if each string is a palindrome and sends 1 (yes) or 0 (no) to Node 3. Node 3 replies "Yes" or "No", and Node 2 prints it. The nodes are connected in two different ways (Fig. 16):

- **Node 1 to Node 2: a topic** (`/rt2string`), because the strings only go one way.
- **Node 2 and Node 3: a service** (`/palindrome_result`), because Node 2 sends something and needs an answer back. This is exactly what a service is for.

B. Code

Node 2 removes spaces, makes the string lowercase, and compares it with its reverse. It then sends the result to Node 3:

```
cleaned = text.replace(' ', '').lower()
result = 1 if cleaned == cleaned[::-1] else 0
req = PalindromeResult.Request(); req.is_palindrome =
     result
fut = self.client.call_async(req)
fut.add_done_callback(lambda f, t=text:
                      self.response_callback(f, t))
```

`call_async` is used, which sends the question and handles the answer when it arrives. If the node stopped and waited for the answer instead, it would freeze and never receive it. Node 3 simply turns the number into a word:

```
def handle_request(self, request, response):
    response.answer = 'Yes' if request.is_palindrome == 1
      else 'No'
    return response
```



Fig. 15. Solution 1 ROS graph: `/wheel_publisher` → `/taskone` → `/speed_subscriber`.

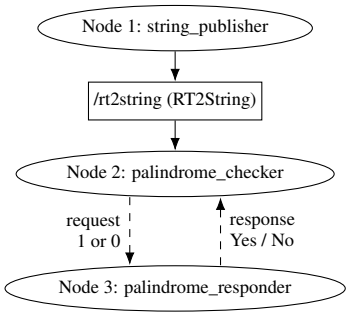


Fig. 16. How the three nodes connect. Solid arrows: topic. Dashed arrows: service.

C. Results

The inputs `AABAA`, `BACAB`, `HELLO` and `racecar` gave Yes, Yes, No and Yes, which are all correct (Fig. 17). In the ROS graph (Fig. 18), Node 3 (`/palindrome_responder`) is not connected to anything. This is expected: `rqt_graph` only draws topics, not services, so the service link between Node 2 and Node 3 does not appear in it.

XIII. Conclusion

ROS 2 Humble was installed and its main parts were learned through the official tutorials. Two solutions were then built using custom messages. The first uses a topic to calculate a wheel's speed, and the second uses a topic and a service together to check palindromes. Using a topic for one-way data and a service for question-and-answer kept both solutions simple. The same ideas are used in a driverless car, where sensors send data on topics, settings are kept in parameters and long tasks are done with actions.

References

[1] Open Robotics, "Quality of Service settings – ROS 2 Humble Concepts." [Online]. Available: https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html

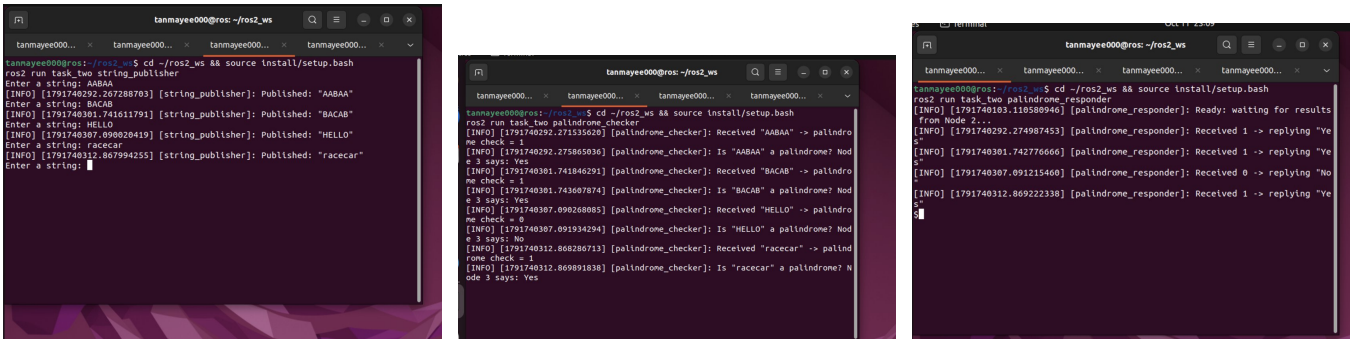


Fig. 17. Solution 2 running: Node 1 sends strings (left), Node 2 checks them and prints Node 3's answer (centre), Node 3 replies (right).

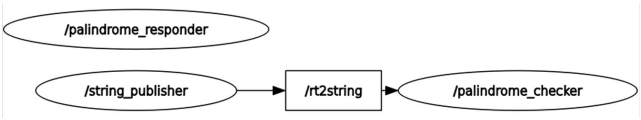


Fig. 18. Solution 2 ROS graph. The service link to `/palindrome_responder` is not drawn by `rqt_graph`.